In-Memory Cache Server (Redis Clone)

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a small in-memory key-value server that speaks the same wire protocol as Redis, supports the most useful subset of Redis commands, keeps all data in RAM, and persists to disk via periodic snapshots. The result is a network server that existing Redis client libraries can connect to without modification: you can open `redis-cli`, connect to your server on its chosen port, and run `SET`, `GET`, `LPUSH`, `INCR`, and the other commands in the supported subset exactly as if you were talking to the real Redis.

Redis is interesting to study because its design is almost the opposite of a traditional relational database. There is no durability for individual writes; there is a single thread doing all the work; there is no query planner, no optimizer, no SQL, and no transactions (in the traditional sense). What Redis does instead is serve requests blindingly fast by keeping everything in memory, avoiding locks through a strictly single-threaded execution model, using hand-tuned in-memory data structures, and pushing persistence to an asynchronous background task. You will build a version of this architecture yourself and see how much of Redis's speed comes not from clever tricks but from the discipline of doing a small number of things well.

The technical content of this project is concentrated in three areas. First, a single-threaded event loop built on `epoll` (Linux) or `kqueue` (macOS), handling many concurrent client connections with one thread and no locks. Second, a parser for the Redis wire protocol (RESP), which is a small, elegant, binary-safe text protocol. Third, a collection of in-memory data structures, strings, lists, hashes, and sorted sets, each implemented from scratch to support its commands efficiently. You will also add a simple snapshot persistence mechanism, so a server restart does not lose all data.

You are not building every feature of Redis. Clustering, replication, scripting (Lua), modules, and the full list of nearly 300 commands are out of scope. You will implement around 25 carefully chosen commands across the four core data types, which is enough to make the server genuinely useful, a running instance will work as a cache in front of a web application, a simple message queue, or a leaderboard backend, exactly as real Redis does.

2 What the Final Product Looks Like

When your project is complete, you will be able to start the server and connect to it using the real `redis-cli` tool that comes with any Redis installation on Linux. The fact that an unmodified client talks to your server unmodified is the defining success criterion of the project.

```
$ make
$ ./minired --port 6380 --data ./dump.rdb
[minired] starting on port 6380
[minired] no snapshot file found, starting empty
[minired] event loop ready

# in another terminal, using the real redis-cli
$ redis-cli -p 6380

127.0.0.1:6380> PING
PONG

127.0.0.1:6380> SET greeting "hello, world"
OK

127.0.0.1:6380> GET greeting
"hello, world"

127.0.0.1:6380> SET counter 0
OK
```

```
127.0.0.1:6380> INCR counter
(integer) 1

127.0.0.1:6380> INCR counter
(integer) 2

127.0.0.1:6380> INCRBY counter 10
(integer) 12

127.0.0.1:6380> LPUSH tasks "wash dishes" "buy bread" "call mom"
(integer) 3

127.0.0.1:6380> LRANGE tasks 0 -1
1) "call mom"
2) "buy bread"
3) "wash dishes"

127.0.0.1:6380> RPOP tasks
"wash dishes"

127.0.0.1:6380> HSET user:1 name "Ayesha" age 27 city "Lahore"
(integer) 3

127.0.0.1:6380> HGET user:1 name
"Ayesha"

127.0.0.1:6380> HGETALL user:1
1) "name"
2) "Ayesha"
3) "age"
4) "27"
5) "city"
6) "Lahore"

127.0.0.1:6380> ZADD leaderboard 100 "Ali" 85 "Bilal" 92 "Hira"
(integer) 3

127.0.0.1:6380> ZRANGE leaderboard 0 -1 WITHSCORES
1) "Bilal"
2) "85"
3) "Hira"
4) "92"
5) "Ali"
6) "100"

127.0.0.1:6380> EXPIRE greeting 60
(integer) 1

127.0.0.1:6380> TTL greeting
(integer) 58

127.0.0.1:6380> SAVE
OK

127.0.0.1:6380> DBSIZE
(integer) 5

127.0.0.1:6380> QUIT
```

The second piece of the final product is that your server keeps working under real load. The benchmark in Phase 3 uses the standard `redis-benchmark` tool (which also ships with Redis) to fire a large number of concurrent requests at your server and measure throughput and latency. Your implementation must sustain at least 50,000 simple operations per second on a modern laptop, which is slower than real Redis but easily within reach of a single-threaded C or C++ implementation that avoids obvious mistakes.

3 The RESP Protocol

Everything the client and server exchange is encoded in RESP, the Redis Serialization Protocol. The protocol is deliberately simple and binary-safe. You need to parse it correctly on the input side and emit it correctly on the output side; the rest of the server does not care about it.

3.1 The Five RESP Types

Every value sent over the wire starts with a single byte indicating its type, followed by the value, followed by `\r\n`. The five types are:

- **Simple String**: `+OK\r\n`. Used for short, successful status replies.
- **Error**: `-ERR unknown command\r\n`. Same format as a simple string but with `-`; the client treats it as an error.
- **Integer**: `:1234\r\n`. A signed 64-bit integer, rendered in ASCII decimal.
- **Bulk String**: `$5\r\nhello\r\n`. A length-prefixed binary-safe string. The length comes first, then `\r\n`, then exactly that many bytes of data, then another `\r\n`. A bulk string can contain any bytes, including zeros and newlines, because the length is explicit. The special form `$-1\r\n` represents a null value (for example, `GET` on a non-existent key).
- **Array**: `*3\r\n...`. A length-prefixed sequence of other RESP values. The length is the number of elements, each of which is itself a RESP value of any type.

3.2 How Commands Are Encoded

Every command from the client is sent as an array of bulk strings. `SET greeting hello` is sent as:

```
*3\r\n
$3\r\nSET\r\n
$8\r\ngreeting\r\n
$5\r\nhello\r\n
```

(Written here with the `\r\n` explicit for clarity; on the wire they are literal CR LF bytes.) This format is uniform: every command, regardless of name or argument count, is an array of bulk strings. The server's parser only needs to know how to read one RESP array containing bulk strings, and from there it can dispatch on the first element (which is the command name).

3.3 The Reply

Different commands return different RESP types. `PING` returns a simple string (`+PONG`). `GET` returns a bulk string or a null bulk string if the key is missing. `INCR` returns an integer. `LRANGE` returns an array of bulk strings. Each command in Section 6 specifies its reply type.

3.4 Parsing Strategy

The parser is a small state machine that reads bytes from the client's input buffer and produces completed RESP values. It must handle partial input gracefully: a TCP `recv` may return only half a command, and the parser needs to remember its position and continue next time more bytes arrive. Do not try to read a complete command in a single `recv` call; instead, each client has a read buffer, `recv` appends to it, and the parser consumes as many complete commands as it can before returning.

This incremental parsing is the single most common source of bugs in the protocol layer. Test it by deliberately chunking input: feed the parser one byte at a time and verify it eventually assembles the same command it would from a single read. Your test suite must include this case.

4 The Single-Threaded Event Loop

Redis achieves its performance through a deliberately simple model: one thread handles everything. All network I/O, all command execution, all data structure manipulation, all persistence triggering — everything happens on a single thread driven by an event loop. There are no locks because there is nothing to lock: only one piece of code is running at any moment.

4.1 Why Single-Threaded

If you have built a multi-threaded server before, the idea of running everything on one thread may sound like a performance disaster. It is not, for two reasons. First, most of the time a server is not doing CPU

work; it is waiting for I/O. A single thread can handle thousands of simultaneous connections as long as it never blocks waiting on any one of them. Second, locking has its own cost: cache line ping-pong between cores, contention on hot data structures, lock-ordering bugs. Removing locks entirely is faster than using fast locks.

The discipline required to make this work is that no handler function may block. Every operation the server performs on a client connection — reading a request, executing the command, writing the reply — must either complete immediately or be broken up so that control returns to the event loop and other clients can be serviced. For this project, the only operations that could possibly block are network reads and writes (handled by making the sockets non-blocking) and disk I/O (handled by doing the snapshot in a forked child process; see Section 7).

4.2 epoll and kqueue

On Linux, the kernel interface for "tell me when any of these file descriptors has I/O to do" is `epoll`. On macOS and BSD, it is `kqueue`. The two APIs are very similar in shape and functionality; write a thin abstraction that exposes whichever one your build target provides. On Linux, the three calls you need are:

```
int epfd = epoll_create1(0);

struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = client_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

struct epoll_event events[MAX_EVENTS];
int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
for (int i = 0; i < n; i++) {
    // events[i].data.fd is ready to read or write
}
```

4.3 Non-Blocking Sockets

Every socket, the listening socket and every client connection, must be set to non-blocking mode using `fcntl(fd, F_SETFL, O_NONBLOCK)`. This means `recv` and `send` return immediately: if there is no data to read, `recv` returns -1 with `errno == EAGAIN` (or `EWOULDBLOCK`, which is the same thing on Linux). If the send buffer is full, `send` returns -1 with the same error. The event loop handles these signals by noting the condition and continuing; when the socket becomes ready again, `epoll` reports it.

4.4 Per-Client Buffers

Each connected client has two buffers: an input buffer for bytes read from the socket and not yet parsed, and an output buffer for bytes to be written to the socket that the kernel has not yet accepted. The event loop's job for each client is roughly:

1. If the client's socket is readable, `recv` into the input buffer; run the RESP parser on the buffer; for each complete command, dispatch it to its handler, which appends the reply to the output buffer.
2. If the client's socket is writable and the output buffer is non-empty, `send` from the output buffer and advance the buffer's start pointer by however many bytes were accepted.
3. If a read or write returns 0 or a fatal error, close the connection and release the buffers.

The event loop dynamically tracks which clients have pending output. When a client's output buffer is empty, the event loop only watches for readable events (`EPOLLIN`); when a client has pending output, the event loop also watches for writable events (`EPOLLIN | EPOLLOUT`). Using `EPOLLOUT` unconditionally wastes CPU because the socket is almost always writable.

5 The Data Types

You will implement four data types. Each is a C struct (or a tagged union) that owns its representation and exposes a small API used by the command handlers.

5.1 The Top-Level Dictionary

The keyspace is a single hash table mapping string keys to *values*, where a value is a tagged union of the four data types plus string. In C:

```
typedef enum {
    OBJ_STRING,
    OBJ_LIST,
    OBJ_HASH,
    OBJ_ZSET
} ObjType;

typedef struct {
    ObjType type;
    union {
        sds           str;     // for OBJ_STRING
        List         *list;    // for OBJ_LIST
        HashTable    *hash;    // for OBJ_HASH
        SortedSet    *zset;    // for OBJ_ZSET
    } ptr;
    int64_t expire_at_ms;      // 0 = no expiration
} Obj;

typedef struct {
    /* hash table mapping sds keys to Obj values */
} Dict;
```

The dictionary grows as items are added. A simple linear-probing or chaining hash table is fine; incremental rehashing in the Redis style is a bonus. The dictionary is the single top-level data structure of your server and is accessed on essentially every command.

5.2 Strings (sds)

Redis strings are binary-safe: they can contain any bytes, including zeros. The standard C `char*` is insufficient because it assumes null termination. Implement a simple string type called `sds` ("simple dynamic string," the same name Redis uses) that stores a length alongside the bytes:

```
typedef struct {
    size_t len;
    size_t cap;
    char data[];  // flexible array member, len bytes of actual data
} sds_header;

typedef char* sds;  // pointer to the data field of an sds_header
```

The trick in Redis is that `sds` is typedef'd to `char*`, but the pointer actually points to the `data` field of the header, with the length and capacity stored in the bytes immediately before. This means an `sds` can be passed to functions expecting `const char*` (they work, assuming no embedded nulls) while still allowing length queries via `sds_len(s)` which reads backward from the pointer. Implement this trick; it is elegant and it demonstrates why people love C.

If you find the negative-offset trick confusing, a simpler alternative is to represent `sds` as a plain struct pointer with explicit `s->data` access everywhere. Both are acceptable; document whichever choice you make.

5.3 Lists

Implement lists as doubly-linked lists of `sds` values. Each node is a struct with a pointer to the previous node, a pointer to the next node, and the value. The list struct holds head and tail pointers and a length counter.

All list commands (`LPUSH`, `RPUSH`, `LPOP`, `RPOP`, `LLEN`, `LRANGE`, `LINDEX`) are straightforward walks of this structure. `LRANGE` with negative indices (like `LRANGE key 0 -1` to get the whole list) requires normalizing the indices first: negative values count backward from the tail, so -1 is the last element

and -2 is the second-to-last.

You do not need to implement the ziplist optimization (Redis's compact representation for small lists). A plain doubly-linked list is sufficient.

5.4 Hashes

A hash (sometimes called a "hash map" to disambiguate from the top-level dictionary) is a mapping from string fields to string values, stored under a single top-level key. Implement it as another instance of your dictionary data structure, with the same hash table code. Redis itself has a special compact representation for small hashes ("listpack"); you do not need to replicate this.

Hash commands (`HSET`, `HGET`, `HDEL`, `HEXISTS`, `HLEN`, `HKEYS`, `HVALS`, `HGETALL`) are all simple wrappers around dictionary operations.

5.5 Sorted Sets

A sorted set is a collection of (member, score) pairs, where members are strings and scores are doubles, and the set is kept ordered by score. Commands include `ZADD` (insert or update), `ZSCORE` (lookup by member), `ZRANGE` (retrieve a range by rank order), `ZRANGEBYSCORE` (retrieve by score range), `ZRANK` (position of a member), and `ZCARD` (size).

Redis implements sorted sets as a pair of structures: a hash table from member to score (for $O(1)$ score lookup) and a skip list keyed by score (for $O(\log n)$ ordered traversal). Implement the same combination. The skip list is the one non-trivial data structure in this project; see Section 5.5.

5.6 Skip Lists

A skip list is a probabilistic alternative to a balanced tree that is much easier to implement correctly. It is a sorted linked list with additional forward pointers at multiple "levels": each node has some random number of forward pointers, with higher-level pointers skipping further ahead. Search starts at the highest level and descends: walk forward at the current level until the next node would overshoot the target, then drop down one level and continue. Insert and delete do the same traversal and then fix up the pointers.

The key property of a skip list is that the expected time for search, insert, and delete is $O(\log n)$, matching a balanced tree, but the code is drastically simpler. A basic skip list implementation is around 200 lines of C.

The level of a new node is chosen randomly: with probability $p = 1/2$, the node has at least one level; with probability p^2, it has at least two; and so on. A reasonable maximum level is 32, which supports up to 2^{32} elements comfortably. On insert, generate a random level for the new node, allocate forward pointers for that many levels, and insert the node into the list at each of those levels.

Pugh's original 1990 paper ("Skip Lists: A Probabilistic Alternative to Balanced Trees," Communications of the ACM) is short, readable, and contains pseudocode for insert, search, and delete. It is the recommended reference. The paper is in the Required Reading list at the end of this manual.

6 The Command Set

You will implement approximately 25 commands. They are listed below, grouped by the data type they operate on. For each command, the semantics are exactly what the Redis documentation at `https://redis.io/commands/` describes; you should open that documentation and cross-reference as you implement each one. The correctness bar is "identical observable behavior to real Redis for the inputs in the test suite," not "close enough."

6.1 Connection and Server

`PING`, `ECHO`, `QUIT`, `SELECT` (accept but ignore, as this project has only one database), `DBSIZE`, `FLUSHDB`.

6.2 Generic Key Operations

`DEL`, `EXISTS`, `TYPE`, `EXPIRE`, `TTL`, `PERSIST`, `KEYS pattern`.

`EXPIRE` sets a time-to-live on a key; after the TTL elapses, the key is automatically deleted. Implement expiration lazily: every time a command reads or writes a key, check whether the key has expired and delete it on the spot if so. A background active-expiration sweep is a bonus, not a requirement.

`KEYS pattern` returns all keys matching a glob pattern. Support `*` (zero or more characters), `?` (exactly one character), and literal characters. A simple recursive match function is sufficient; full glob support with character classes is out of scope.

6.3 String Operations

`SET`, `GET`, `SETNX`, `APPEND`, `STRLEN`, `INCR`, `INCRBY`, `DECR`, `DECRBY`.

`INCR` and friends must parse the current value as an integer, increment it, and write the result back. If the key does not exist, treat it as 0. If the key exists but its value is not a valid integer string, return an error.

6.4 List Operations

`LPUSH`, `RPUSH`, `LPOP`, `RPOP`, `LLEN`, `LRANGE`, `LINDEX`.

6.5 Hash Operations

`HSET`, `HGET`, `HDEL`, `HEXISTS`, `HLEN`, `HKEYS`, `HVALS`, `HGETALL`.

6.6 Sorted Set Operations

`ZADD`, `ZSCORE`, `ZRANGE` (with optional `WITHSCORES`), `ZRANK`, `ZCARD`, `ZREM`.

7 Persistence: Snapshots

Without persistence, every restart loses all data. Implement a simple snapshot mechanism based on forking.

7.1 The Snapshot Format

A snapshot file is a single binary file containing every key and value currently in the dictionary. Use the following format:

```
Offset  Size   Field            Description
------  -----  --------------   --------------------------------------
  0      4     magic            ASCII bytes 'M','R','D','B'
  4      4     version          uint32, must be 1
  8      8     key_count         uint64, number of top-level keys

For each key, in any order:
  1      type             uint8 (0=string, 1=list, 2=hash, 3=zset)
  8      expire_at_ms     int64 (0 if no expiration)
  4      key_len          uint32
  var    key bytes
  var    type-specific body (described below)
```

Type-specific bodies:

- **String**: 4-byte length followed by the bytes.
- **List**: 4-byte element count followed by, for each element, a 4-byte length plus its bytes.
- **Hash**: 4-byte entry count followed by, for each entry, a 4-byte field length, the field bytes, a 4-byte value length, and the value bytes.
- **Sorted set**: 4-byte entry count followed by, for each entry, an 8-byte score (as IEEE 754 double in native byte order), a 4-byte member length, and the member bytes.

At the end of the file, append an 8-byte CRC64 of everything written so far. On load, verify the CRC before using any of the data.

7.2 Writing the Snapshot with fork

Here is the elegant trick that makes Redis's snapshotting work without blocking the event loop. When the user issues the `SAVE` command (or the server decides to snapshot automatically), the server calls `fork()`. The child process inherits a copy-on-write snapshot of the parent's memory at the moment of the fork. The child's job is simple: write every key in its (frozen) copy of the dictionary to a temporary file, `rename()` it into place, and exit. The parent continues running the event loop normally while the child writes; any writes the parent makes during this time modify pages that are copied on write, leaving the child's view untouched.

In practice this means the `SAVE` command handler:

1. Calls `fork()`.
2. If the return value is 0, you are in the child: iterate the dictionary, serialize to `dump.rdb.tmp`, `fsync`, `rename()` to `dump.rdb`, exit 0.
3. If the return value is positive, you are in the parent: note the child's pid so that a future `SIGCHLD` handler can reap it, and reply `OK` to the client immediately.
4. If the return value is negative, fork failed; reply with an error.

This is one of the few places in the project where you will use an OS primitive (`fork`) that does something genuinely unusual. Take a moment to understand copy-on-write before implementing it; the `fork(2)` manual page and any Linux systems programming textbook will explain it adequately.

7.3 Loading on Startup

On startup, check whether the snapshot file exists. If so, read it, verify the magic and version, rebuild the in-memory dictionary by deserializing every entry, verify the CRC at the end, and report the load result. If the CRC does not match, or if any record's length is inconsistent with the file, abort startup with an error; do not try to recover partial data.

8 Phase 1 — Event Loop, RESP, and Strings

The goal of Phase 1 is a working server that accepts concurrent clients, parses RESP correctly, and implements enough commands to be tested with `redis-cli`: `PING`, `ECHO`, `SET`, `GET`, `DEL`, `EXISTS`, `DBSIZE`, `QUIT`. Only the string data type is implemented. No persistence, no expiration, no other commands.

Build these components:

- The `sds` dynamic string type, with constructors, length, append, and free.
- The top-level dictionary as a hash table from `sds` key to a tagged value. For Phase 1, the only tagged value type is `OBJ_STRING`.
- The `epoll` (or `kqueue`) event loop with non-blocking sockets. The listening socket is registered on startup; each accepted client is added to the loop.
- Per-client input and output buffers; `EPOLLIN` is always watched, `EPOLLOUT` is only watched when the output buffer is non-empty.
- A complete RESP parser that handles incremental input. Unit test it by feeding single bytes.
- A complete RESP writer that produces each of the five reply types.
- The command dispatch: given a parsed command (an array of bulk strings), look up the first element in a command table, and call the handler with the remaining arguments.
- Handlers for `PING`, `ECHO`, `SET`, `GET`, `DEL`, `EXISTS`, `DBSIZE`, `QUIT`.

At the end of Phase 1, `redis-cli -p 6380` must work for all eight commands. Run `redis-benchmark -p 6380 -t set,get` and verify that it completes without errors; report the observed throughput.

9 Phase 2 — Full Command Set and Expiration

The goal of Phase 2 is to implement all remaining commands (Section 6) and the TTL mechanism.

Build these components:

- The list, hash, and sorted set data types (Sections 5.3, 5.4, 5.5). The skip list is the only non-trivial component; budget a full day for it alone and test it in isolation before wiring it up.
- All commands listed in Section 6 that were not already done in Phase 1.
- Lazy expiration: before reading or writing any key, check `expire_at_ms` against the current time and delete the key if expired.
- The `EXPIRE`, `TTL`, and `PERSIST` commands.
- The `KEYS pattern` command with a glob-style matcher.
- The `TYPE` command, which inspects the tagged value and returns the type name.

At the end of Phase 2, the example session in Section 2 should work end to end (except for `SAVE`, which is Phase 3). Every command in Section 6 must produce the same observable result as real Redis for reasonable inputs.

10 Phase 3 — Persistence and Benchmark

The goal of Phase 3 is the snapshot mechanism and the benchmark.

10.1 Snapshots

Implement the snapshot writer and reader per Section 7, using `fork()` as described. Add the `SAVE` command (synchronous: wait for the child to finish before replying) and the `BGSAVE` command (asynchronous: reply immediately and let the child write in the background). Add automatic snapshot loading on startup.

10.2 Benchmark

Use the real `redis-benchmark` tool (which ships with any Redis installation: `sudo apt install redis-tools`). The benchmark will open many concurrent connections to your server and fire a large number of commands through each.

Run the benchmark in at least these configurations and record the results:

- `redis-benchmark -p 6380 -t ping -n 100000` — baseline throughput with the simplest possible command.
- `redis-benchmark -p 6380 -t set,get -n 100000 -r 10000` — string operations with 10,000 distinct keys.
- `redis-benchmark -p 6380 -t lpush,lpop -n 50000` — list operations.
- `redis-benchmark -p 6380 -t hset -n 50000` — hash operations.
- `redis-benchmark -p 6380 -t zadd -n 50000` — sorted set operations. This exercises the skip list.

For each run, record the throughput in operations per second and the 50th / 95th / 99th percentile latencies (which `redis-benchmark` prints by default). Include these numbers in your design document along with a brief analysis of which commands are fastest, which are slowest, and why.

A successful submission sustains at least 50,000 operations per second on `SET`/`GET`, which is a safe target for a single-threaded implementation on a modern laptop. If your numbers are significantly lower, spend time profiling before submitting: the usual culprits are unnecessary allocations in the hot path, excessive copying of buffers, and calling `recv`/`send` too many times with tiny sizes.

10.3 Persistence Correctness Test

In addition to the performance benchmark, add a persistence correctness test:

1. Start the server with an empty data directory.
2. Insert 10,000 key-value pairs of each data type (strings, lists, hashes, sorted sets).
3. Call `SAVE`.
4. Shut down the server.
5. Restart the server pointing at the same data directory.
6. Verify that every inserted key is present and has the same value it had before shutdown.

This test protects against bugs in the snapshot format and in the load path. A regression in either of them is immediately visible.

11 Deliverables and Submission

11.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation.
- A `Makefile` (or `CMakeLists.txt`) that builds the `minired` binary with a single command on a recent Ubuntu LTS.
- A `README.md` with build instructions, a list of implemented commands, instructions for connecting with `redis-cli`, and any known limitations.
- A `design.pdf` document of at most 10 pages explaining: the event-loop architecture, the RESP parser's incremental-input handling, the top-level dictionary, each of the four data type implementations (especially the skip list), the snapshot format, the `fork`-based persistence, the lazy-expiration

mechanism, and a discussion of the benchmark results with your throughput numbers and latency percentiles.
- A `benchmark/` directory with the shell script that runs the `redis-benchmark` suite against a freshly started server and the output of your final run.
- A `tests/` directory with tests covering: the RESP parser (especially partial-input cases), the `sds` string type, each data type in isolation, the skip list in isolation, the snapshot round-trip (save, reload, verify), and the TTL / expiration behavior.

11.2 Archive Naming

`Group[Number]_Project02_Redis.zip`, for example `Group17_Project02_Redis.zip`.

11.3 Demonstration

Each group will present a 20-minute live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a brief walkthrough of the architecture using the design document. Second, a live connection using `redis-cli` with the instructor free to type commands on the spot and observe the server's behavior, followed by running the benchmark against a freshly compiled server. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation, with particular focus on the event loop's non-blocking discipline and the `fork`-based snapshot trick. All group members must be present. Missing the demonstration results in a zero for Phase 3.

12 Bonus Opportunities

Each of the following is independent. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points. To claim bonus credit, your design document must include a dedicated section explaining which items you attempted and how you verified them.

12.1 AOF Persistence (up to 5%)

In addition to snapshots, implement Append-Only File persistence: every write command is appended to a log file (in RESP format, as if the client had sent it). On startup, the server replays the log by running each command as if a client had sent it, producing the exact same state as before shutdown. Support the three `fsync` policies Redis offers: `always` (fsync after every command, safest), `everysec` (fsync once per second, the Redis default), and `no` (never fsync explicitly, fastest). Benchmark the tradeoff.

12.2 Pub/Sub (up to 5%)

Implement the Redis publish/subscribe mechanism. Commands: `SUBSCRIBE channel`, `UNSUBSCRIBE`, `PUBLISH channel message`. When a client is subscribed to one or more channels, a `PUBLISH` on one of those channels causes the server to push the message to every subscribed client. Subscribed clients cannot issue other commands until they unsubscribe. Verify the behavior with two `redis-cli` instances: one subscribed, one publishing.

12.3 Active Expiration Sweep (up to 5%)

The base project uses lazy expiration, which leaves expired keys sitting in memory until the next time they happen to be accessed. Add a background sweep that runs once per second and actively expires a sample of keys, mirroring the Redis behavior described in its `expires.c` source file. The trick is to keep the sweep bounded in cost so it does not block the event loop for too long. Describe your sampling strategy in the design document.

13 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of Redis itself, or any embeddable cache library (hiredis, memcached, KeyDB, Dragonfly, Valkey) as a component of your server. The point of the project is to build the server yourself.
- Use of any existing RESP parser or serializer library. Write your own.
- Use of any existing data structure library that provides skip lists, sorted maps, or priority queues (GLib, Abseil, folly, etc.) for the sorted set implementation. The skip list in particular must be your own code.

- Use of a multithreaded architecture with locks for the command execution path. Redis is single-threaded by design and this project follows suit. Background workers for snapshotting via `fork` are fine; threads sharing the dictionary are not.
- Use of text-based storage (JSON, CSV, XML) as the snapshot format. The snapshot must be the binary format described in Section 7.
- Use of `select` or `poll` instead of `epoll` or `kqueue` for the event loop. `select` has a hard limit of 1024 file descriptors; `poll` is linear in the total number of clients. `epoll`/`kqueue` are the modern, scalable APIs and are what Redis itself uses.
- Use of Python at any layer, including test harnesses. Tests and benchmarks must be written in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. The incremental nature of a server of this size should be visible in your git log.

14 Required Reading

The first two items are short and required reading before you start. The remaining items are reference material to consult during implementation.

1. Redis documentation, "Redis protocol specification" at `https://redis.io/docs/reference/protocol-spec/`. The complete RESP specification in a few pages. Read it before Phase 1.
2. Pugh, W. "Skip Lists: A Probabilistic Alternative to Balanced Trees." *Communications of the ACM*, 33(6):668–676, 1990. The original skip list paper. Short, readable, contains pseudocode. Read it before Phase 2.
3. Carlson, J. L. *Redis in Action*, Manning, 2013. A practical book covering Redis from a user's perspective. Useful for understanding what commands are supposed to do and why the data types are designed the way they are. Chapter 2 is particularly relevant.
4. The Redis source code, specifically the files `src/networking.c` (for the event loop and client management), `src/sds.c` (for the `sds` type), `src/t_zset.c` (for the sorted set and skip list), and `src/rdb.c` (for the snapshot format). The Redis codebase is famously clean C and is well worth reading as reference material. You may not copy from it, but studying it to understand the architecture is encouraged.
5. Redis documentation for individual commands at `https://redis.io/commands/`. The authoritative reference for what each command in Section 6 is supposed to do. Keep this open while implementing each handler.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday, 11:30 AM – 2:30 PM